

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	V Sunil Kumar
Reg. No	192425292

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I V Sunil Kumar/192425292 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: V Sunil Kumar

QUESTIONS:05

Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer :

Gradient Descent updates the model parameters using:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

)

where:

- ☐ θ = model parameters
- ☐ η = learning rate
- ☐ $J(\theta)$ = loss function

Effect of a High Initial Learning Rate

A high initial learning rate allows the optimizer to take large steps toward the minimum.

Advantages:

- ☐ Faster initial convergence.
- ☐ Quickly escapes flat regions or shallow local minima.
- ☐ Reduces training time in early iterations.

As training progresses, the learning rate is gradually decreased (learning rate decay), resulting in:

- ☐ Smaller parameter updates.
- ☐ Stable convergence.
- ☐ Better fine-tuning near the optimum.

When It Outperforms a Constant Learning Rate

A decaying learning rate performs better when:

- ☐ The loss function is smooth and convex.
- ☐ The initial point is far from the optimum.
- ☐ Fast exploration is needed initially and precise convergence later.

A constant learning rate:

- ☐ May converge slowly if too small.
- ☐ May oscillate around the optimum if too large.

Risks

- ☐ Very high learning rate may overshoot the global minimum.
- ☐ Can cause oscillations or divergence.
- ☐ Training may become unstable.

Conclusion:

Using a high initial learning rate followed by decay combines fast learning with stable convergence, often outperforming a constant learning rate when the decay schedule is properly chosen.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases

What does this imply about the robustness of MLE?

Answer:

Derivation of MLE for Mean (μ)

Assume the data $x_1, x_2, \dots, x_n$ follows a Gaussian distribution:

$$X_i \sim N(\mu, \sigma^2)$$

where μ is **unknown** and σ^2 is **known**.

The likelihood function is:

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Taking the log-likelihood:

$$\log L(\mu) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Differentiate with respect to μ and set it to zero:

$$\frac{d}{d\mu} \log L(\mu) = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0$$

Hence,

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

Therefore, the **Maximum Likelihood Estimator (MLE)** of μ is the **sample mean**.

Effect of Outliers

- The sample mean is highly sensitive to extreme values.
- A few large outliers can shift the estimated mean significantly.
- Therefore, MLE for Gaussian data is **not robust** to outliers.

Effect of Increasing Sample Size

- As the number of samples increases, the estimate becomes more accurate.
- The variance of the estimator decreases.
- The estimated mean converges to the true mean (Law of Large Numbers).

Implication on Robustness

- MLE provides efficient estimates when data follows a Gaussian distribution.
- However, it is **sensitive to noisy data and outliers**.

- Robust estimators such as the **median** or **Huber estimator** perform better when outliers are present.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Challenges

Given:

- Features = 10,000
- Training samples = 500

Challenges include:

- High-dimensional data.
- Overfitting due to limited samples.
- Increased computational cost.
- Presence of irrelevant and redundant features.

Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize features.

Step 2: Feature Selection

- Remove irrelevant features using statistical tests or L1 (Lasso) regularization.
- Keep only informative features.

Step 3: Dimensionality Reduction

- Apply Principal Component Analysis (PCA).
- Reduce 10,000 features to a smaller set while preserving important information.

Step 4: Regularization

- Train models with L1 or L2 regularization.
- Prevent large model weights and reduce overfitting.

Step 5: Model Training

- Use algorithms such as Support Vector Machine (SVM) or Logistic Regression with regularization.

Step 6: Cross-Validation

- Perform k-fold cross-validation.
- Select the best model and hyperparameters.

How It Prevents Overfitting

- Feature selection removes unnecessary variables.
- PCA reduces dimensionality and noise.
- Regularization limits model complexity.
- Cross-validation ensures good performance on unseen data.

Thus, the model achieves better generalization and improved prediction accuracy.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

Why More Hidden Neurons Increase Representational Power

An MLP with sigmoid activation learns nonlinear relationships.

Adding more hidden neurons allows the network to:

- Learn more complex decision boundaries.
- Capture intricate feature interactions.
- Approximate complicated nonlinear functions.

According to the **Universal Approximation Theorem**, a neural network with enough hidden neurons can approximate almost any continuous function.

Why More Neurons Do Not Always Improve Performance

Increasing hidden neurons also increases:

- Number of trainable parameters.
- Model complexity.
- Training time.

If the model becomes too complex, it may memorize the training data instead of learning general patterns.

This causes **overfitting**, where:

- Training accuracy becomes very high.
- Test accuracy decreases.

Generalization

A good model should perform well on unseen data.

Better generalization is achieved by:

- Using regularization.
- Applying dropout.
- Early stopping.
- Cross-validation.
- Choosing an appropriate number of hidden neurons.
-

Conclusion:

More hidden neurons improve learning capability, but excessive neurons increase the risk of overfitting. The optimal network balances **representational power** and **generalization**.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

Impact on Gradient Updates

In high-dimensional datasets:

- Many features contribute little useful information.
- Gradients become noisy and unstable.
- Some gradients become very small (vanishing gradients).
- Learning becomes inefficient.

Impact on SGD Convergence

Stochastic Gradient Descent:

- Requires more iterations.
- Experiences slower convergence.
- May oscillate due to noisy gradients.
- Needs careful learning-rate tuning.

Impact on Model Generalization

High-dimensional data often contains:

- Redundant features.
- Irrelevant variables.
- Sparse samples.

This increases overfitting, causing poor performance on unseen data.

Techniques to Overcome These Issues

1. Dimensionality Reduction (PCA)

- Removes redundant features.
- Reduces computational complexity.
- Produces more stable gradients.
- Improves convergence speed.

2. Regularization (L1/L2, Dropout)

- Penalizes large weights.
- Prevents overfitting.
- Improves model generalization.

Additional Techniques

- Feature Selection: Keeps only informative features.
- Batch Normalization: Stabilizes gradients and accelerates training.
- Early Stopping: Stops training before overfitting occurs.

Conclusion:

The curse of dimensionality slows SGD, produces unstable gradient updates, and reduces generalization. Applying dimensionality reduction, regularization, and feature selection improves convergence, reduces overfitting, and enhances prediction performance.